

Disciplina: Estrutura de Dados

Professor(a): Wellington Gadelha

Sistema de Gerenciamento

1. OBJETIVO

Desenvolver um Sistema de Gerenciamento de Mercadinho com Interface Gráfica em linguagem C pura utilizando a biblioteca Raylib. O projeto deve integrar de forma harmônica a renderização visual em tempo real com o uso de estruturas de dados lineares, não-lineares, algoritmos de ordenação e persistência em arquivos.

2. INTERFACE

O sistema não rodará mais no terminal nativo. Ele deverá abrir uma janela gráfica gerenciada pela Raylib. Toda a interação do usuário (operador do mercadinho) ocorrerá por meio de componentes visuais renderizados na tela (menus dinâmicos, caixas de texto simuladas, tabelas e botões clicáveis).

- Requisito de Inicialização: Ao iniciar a janela, o sistema deverá carregar os dois arquivos de texto para popular a memória:
 - produtos.txt (1.000 registros): Base de dados para popular a Tabela Hash.
 - clientes.txt (1.000 registros): Lista de nomes e datas para popular a Fila Dinâmica.

Cada registro deve conter obrigatoriamente o campo de Data de Registro (dia, mes, ano) para fins de auditoria.

3. REQUISITOS

Fica estritamente proibido o uso de coleções nativas ou arrays estáticos globais de tamanho fixo para as coleções principais. Toda a lógica de ponteiros e gerenciamento de memória deve ser manual.

- Tabela Hash (Estoque): Array de tamanho primo com Encadeamento Externo (lista encadeada para colisões). Modificado via interface ao cadastrar produtos.
- Fila Dinâmica (Atendimento): Fila baseada em ponteiros. A interface da Raylib deve exibir visualmente o "Próximo Cliente" e, se possível, o número de clientes restantes na fila.
- Pilha Dinâmica (Carrinho de Compras): Utilizada no momento da venda. O operador digita o código do produto na interface; o sistema busca na Hash e empilha no carrinho. Deve haver um botão visual "Desfazer (Undo)", que desempilha o último item e estorna o estoque.

Disciplina: Estrutura de Dados

Professor(a): Wellington Gadelha

- Árvore Binária de Busca - BST (Histórico): Cada venda finalizada com sucesso insere um nó na BST (indexado pelo id_venda).

O módulo de Relatório de Faturamento deve ser exibido de forma organizada na tela da Raylib (em formato de lista rolável ou paginação simples):

- Ao clicar no botão "Relatório por Faturamento", o sistema deve extrair os dados da BST para um vetor alocado dinamicamente (malloc).
- Aplicar manualmente um algoritmo de ordenação (ex: *Quick Sort*, *Merge Sort* ou *Insertion Sort*) para ordenar o vetor pelo valor_total de forma decrescente.
- Renderizar na tela a lista ordenada (Ranking das maiores vendas do dia), mostrando ID, Valor e Data. Liberar a memória do vetor com free() ao fechar o relatório.

4. FUNCIONALIDADES

O sistema deverá gerenciar o fluxo do mercadinho através de uma interface gráfica estruturada nas seguintes funcionalidades principais:

- Tela: Menu Principal
 - Navegação por Botões: Renderização de botões clicáveis com o mouse para acessar os diferentes módulos do sistema (Caixa/Atendimento, Cadastro de Produtos, Relatórios e Consulta).
 - Dashboard de Status: Exibição em tempo real do estado atual das estruturas de dados na memória (ex: "Clientes aguardando na fila: X", "Total de produtos cadastrados no estoque: Y").
- Tela: Cadastro de Produtos
 - Carga Automatizada (Inicialização): Leitura obrigatória do arquivo produtos.txt para popular a Tabela Hash com os 1.000 registros iniciais assim que o programa inicia.
 - Cadastro Manual de Novos Itens: Interface com campos de texto simulados para o operador inserir manualmente novos produtos informando: Código, Nome, Preço, Quantidade em Estoque e Data de Cadastro.
 - Inserção Resiliente: Processamento do código através da função Hash e inserção correta na estrutura, tratando colisões por encadeamento externo (lista encadeada).

Disciplina: Estrutura de Dados

Professor(a): Wellington Gadelha

- Tela: Frente de Caixa e Atendimento
 - Carga Automatizada de Clientes: Preenchimento automático da Fila Dinâmica no início do sistema com os 1.000 clientes extraídos de clientes.txt.
 - Chamada de Próximo Cliente: Botão visual para desenfileirar o próximo cliente da fila e iniciar o atendimento.
 - Bipe de Itens (Carrinho de Compras): Campo para digitar o código do produto. O sistema busca o item na Tabela Hash, decrementa 1 unidade do estoque e empilha o produto na Pilha Dinâmica do carrinho atual.
 - Funcionalidade Estorno (Undo): Um botão visual "Desfazer Último Item" que desempilha o último produto passado, cancelando-o do carrinho e devolvendo automaticamente a quantidade estornada para o estoque na Tabela Hash.
 - Finalização de Venda (Checkout): Botão para encerrar a compra. O sistema descarrega a pilha do carrinho para calcular o valor total, registra a transação com a data atual e insere um novo nó contendo a venda na Árvore Binária de Busca (BST).
- Tela: Relatórios Gerenciais
 - Relatório Cronológico/ID: Exibe na tela a listagem de todas as vendas realizadas. A listagem deve ser gerada através do caminhamento *In-Order* (Em-Ordem) diretamente sobre a árvore, exibindo os dados ordenados pelo ID da venda.
 - Relatório de Faturamento (Algoritmo de Ordenação - M3): Botão que extrai dinamicamente todos os nós da árvore para um vetor alocado com malloc. O sistema aplica manualmente um algoritmo de ordenação (ex: *Quick Sort* ou *Merge Sort*) sobre o campo valor_total. A tela deve renderizar um Ranking Decrescente (do maior faturamento para o menor) contendo ID, Valor e Data da Venda. A memória do vetor deve ser liberada com free() ao sair da tela.
- Tela: Auditoria e Consulta
 - Busca de Vendas por ID: Campo de texto onde o usuário digita o número de um cupom fiscal (ID da Venda). O sistema realiza a busca binária navegando pelos nós da árvore e exibe na tela os detalhes daquela venda específica (produtos, valor total e data) ou uma mensagem de "Venda não encontrada".

Disciplina: Estrutura de Dados

Professor(a): Wellington Gadelha

5. CRITÉRIOS DE AVALIAÇÃO

- Adequação do Tema e UX/UI (1,5 pontos): Implementação correta das estruturas e capacidade de integrar o loop da Raylib com a lógica de ponteiros. A interface deve ser funcional (cliques de mouse funcionando e renderização correta de strings).
- Clareza, Organização e Vazamento de Memória (3,5 pontos): Organização do código (divisão clara entre funções de renderização e funções lógicas de ED).
- Domínio do Conteúdo e Apresentação Oral (5,0 pontos): Defesa por grupo em laboratório. Os grupos deverão explicar conceitos de ordenação/árvores e modificar parâmetros visuais ou lógicos (ex: alterar a cor de um botão ou o critério de busca) na hora para comprovar a autoria.

6. ENTREGA

Para a entrega final, os grupos deverão consolidar todos os arquivos em único arquivo compactado no formato .zip ou .rar. O arquivo deve ser enviado exclusivamente através do link da atividade no Ambiente Virtual de Aprendizagem (AVA) dentro do prazo estabelecido.